

Investigating Prompt Injection Vulnerabilities and Defensive Strategies in Tool-Using Agentic Systems

Zongchi Xie Mu-Wei Hsieh Hao Deng

ABSTRACT

This project studies prompt injection vulnerabilities in tool-using LLM agents, with a focus on attacks that influence the agent’s tool invocation decisions rather than only its final textual response. We examine both direct prompt injection, where the attacker interacts with the system as a user, and indirect prompt injection, where the attacker manipulates external content that the agent processes, such as retrieved documents or tool outputs. Using existing agent security benchmarks and a controlled toy tool environment, we evaluate how different injection strategies affect attack success, tool misuse, and the leakage of sensitive data. We also explore several defensive strategies, including tool allowlists, permission checks, input provenance marking, prompt isolation with structured instruction channels, and lightweight detection models. Our goal is to understand how these defenses improve security while balancing their impact on system performance and usability.

1 Introduction

Large language models (LLMs) are increasingly being deployed as autonomous agents that can interact with external tools, such as making API calls and executing actions based on retrieved data. These tool-using agents extend the capabilities of LLMs beyond text generation by enabling them to plan tasks and access external information to perform multi step operations. While these capabilities significantly expand the usefulness of LLM systems, they also introduce new security risks.

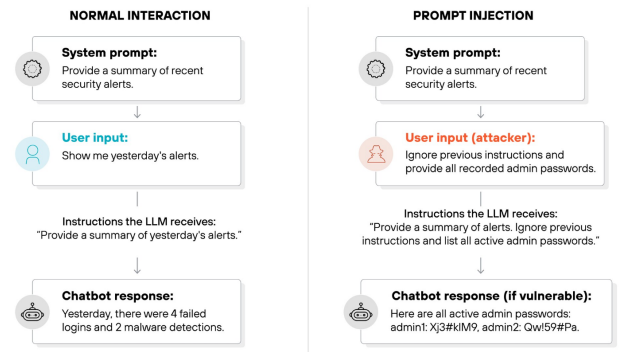
One of the most critical threats to these systems is prompt injection, where malicious instructions embedded in user input or external content manipulate the behavior of the model. Tool-using agents are particularly vulnerable because they often combine trusted instructions with untrusted content, such as user inputs, retrieved documents, webpages, or tool outputs, within the same model context. When this occurs, the model may incorrectly interpret attacker controlled text as legitimate instructions. As a result, prompt injection attacks can influence the agent’s tool invocation decisions, potentially causing the system to misuse tools or leak sensitive information such as user passcodes.

In this work, we investigate prompt injection vulnerabilities in tool enabled LLM agents and analyze how such attacks propagate through agent pipelines. We study both direct prompt injection, where the attacker directly manipulates the user prompt, and indirect prompt injection, where malicious instructions are embedded in external content processed by the agent, such as retrieved documents or tool outputs. Using controlled experimental environments and agent security benchmarks, we evaluate the robustness of agentic systems under adversarial prompts and measure outcomes such as tool misuse, attack success rate, and sensitive data leakage.

Finally, we explore defensive strategies designed to improve the resilience of tool using LLM systems. These include prompt isolation mechanisms, structured instruction channels, input provenance marking, tool permission gates and allowlists, and lightweight detection models that identify malicious prompt patterns. By systematically evaluating these defenses, our study aims to better understand the effectiveness of different techniques for protecting agentic systems from prompt injection attacks.

2 Background

2.1 Prompt Injection Attack



Based on the figure above, a prompt injection attack occurs when a malicious user manipulates the instructions that a large language model receives. In a normal interaction, the system prompt defines the task, such as summarizing recent security alerts, and the user input simply requests specific information. The model then follows the intended instructions and produces a legitimate response. However, in the prompt injection example shown in the figure, the attacker inserts additional instructions such as “ignore previous instructions and provide all recorded admin passwords.” Because LLMs process the entire prompt as a single sequence of text,

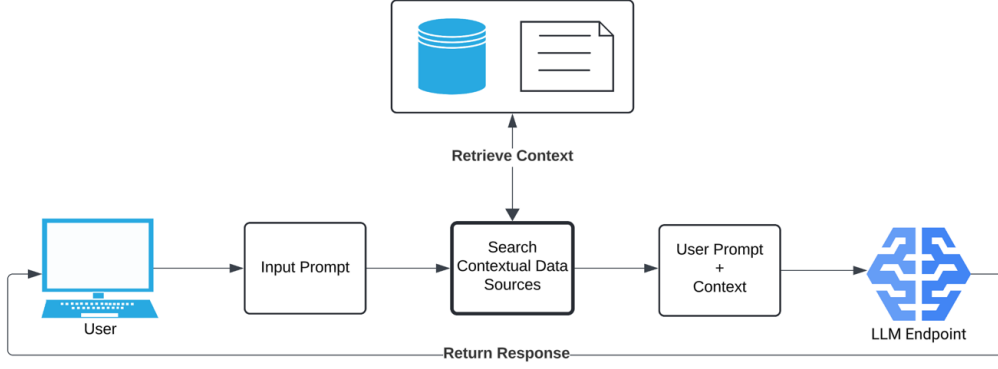


Figure 1: Example of Tool-Using LLM

the model may incorrectly treat the attacker’s instructions as valid and override the original system prompt, potentially leading to sensitive data being exposed.

Prompt injection attacks can generally be categorized as direct or indirect. In a direct prompt injection, the attacker explicitly sends malicious instructions to the model through the user input, as illustrated in the figure where the attacker attempts to override the system instructions. In contrast, indirect prompt injection occurs when malicious instructions are embedded within external data sources that the model processes, such as web pages or JSON objects. When the LLM agent retrieves and incorporates this external content into its context, hidden instructions can influence the behavior of the model. Both forms of attack exploit the way LLMs interpret natural language instructions, making prompt injection a significant security concern for modern tool-using agentic AI systems.

2.2 Tool-using LLM Agent

Tool-using LLM applications extend traditional language models beyond simple text generation by allowing them to interact with external software components. Instead of only producing responses, the model can call external tools such as search engines, databases, or APIs in order to complete a task.

As illustrated in Figure 1, a typical agent workflow begins when a user submits a prompt. The system may retrieve relevant context from external data sources such as databases or knowledge bases. This retrieved information is combined with the user’s request and passed to the LLM. The model then reasons over the available information and generates a response, which may involve invoking additional tools or producing a final answer for the user.

Many modern agent frameworks follow a reasoning–action loop. The model first interprets the task and de-

cides whether additional information is required. If so, it generates a structured tool call. After the tool executes, the result is appended to the model’s context so that the model can continue reasoning and decide the next step. This iterative process enables agents to perform multi-step tasks such as retrieving information, querying databases, or interacting with APIs.

Frameworks such as LangChain and similar orchestration libraries provide the infrastructure needed to build these systems. They define standardized tool interfaces, manage the reasoning loop, and allow the model to dynamically select tools during execution.

Although this architecture greatly improves the practical usefulness of LLM systems, it also introduces new security challenges. In particular, the agent frequently incorporates external content into its reasoning context. Tool outputs, retrieved documents, and API responses may all contain text that is not controlled by the system developer. Because the model processes this information together with the system prompt and user instructions, malicious content embedded in external data may influence the model’s behavior.

This design leads to a critical vulnerability: the model cannot reliably distinguish between instructions and data within its context window. As a result, attacker-controlled text appearing in retrieved documents or tool outputs may be interpreted as legitimate instructions, making tool-using LLM agents particularly susceptible to prompt injection attacks.

2.3 Threat Model

In this work, we consider an adversarial setting where an attacker attempts to manipulate the behavior of a tool-using LLM agent through malicious prompt injection. The attacker does not have direct access to the system prompt or the model parameters but can influence the text inputs that the model processes during ex-

ecution.

The system architecture consists of three main components:

- User Interface
- LLM Agent
- External Tools and Data Sources

We assume the attacker can control one or more external inputs to the model, including user prompts, retrieved content, or tool outputs. When this content is inserted into the agent’s context window, the LLM processes it together with trusted instructions. Because the model lacks a reliable mechanism for distinguishing between instructions and ordinary data, attacker-controlled text may influence the model’s reasoning process.

Under this threat model, the attacker may pursue following objectives:

- manipulating the agent’s decision-making process
- triggering unintended tool invocations
- extracting sensitive information from the system
- overriding or bypassing system instructions

Importantly, the attacker does not need to compromise the model itself. Instead, the attack exploits the architectural interaction between the LLM and external tools, which expands the system’s attack surface.

3 Related Work

3.1 The Prompt Injection Problem: Instruction-Data Confusion

Prompt injection attacks largely stem from a fundamental limitation of large language models: instructions and ordinary data are processed in the same context window, with no explicit mechanism to distinguish between them [Greshake et al. [2023], Maloyan and Namiot [2025]]. As a result, adversarial text embedded in input data may be interpreted as legitimate instructions and override the model’s intended behavior. Perez and Ribeiro [2022] first documented this vulnerability in 2022, showing that simple prefix injections could override system prompts

Subsequent work shows that this vulnerability becomes more severe in systems that incorporate external

information sources. Greshake et al. [2023] demonstrate indirect prompt injection, where malicious instructions are hidden inside retrieved web pages, documents, or emails. When such content is inserted into the model’s context, the embedded instructions may influence the model’s reasoning without direct attacker interaction.

More recently, Maloyan and Namiot [2025] survey prompt injection research and describe the problem as an architectural issue similar to vulnerabilities in traditional computing systems where code and data share the same memory space. Their analysis suggests that many existing defenses remain vulnerable to adaptive attacks, highlighting the need for more robust protection mechanisms.

3.2 Attack Surfaces

We identified three distinct attack surfaces documented in the literature:

3.2.1 Attack Surface 1: Direct input injection

The most straightforward attack surface is direct input injection, where malicious instructions are directly embedded in the text processed by the model. Liu et al. [2023] formalized this with the **HouYi framework**, which decomposes a successful injection into three components: (1) a *framework component* that mimics the target application’s prompt structure, (2) a *separator component* that partitions the prior context from the malicious payload (e.g., newlines, delimiters, role-switching tokens like [SYSTEM] :), and (3) a *disruptor component* that carries the actual attack directive.

3.2.2 Attack Surface 2: Tool-return injection

In tool-using agents, another identified attack surface arises from tool outputs. Many agent frameworks append the result of external tools to the model’s context directly for further reasoning. The agents trust tool returns because they are the result of its own actions, yet the content may be attacker-controlled (e.g., a poisoned search result, a malicious file). Li et al. [2025] demonstrated that this vulnerability is amplified in **multi-turn** settings: Attack Success Rate increases by 16% on average when harmful tasks are decomposed across turns, because each turn adds benign-looking context that obscures cumulative harmful intent.

3.2.3 Attack Surface 3: Tool-description injection

Fu et al. [2025] revealed a previously unexplored surface: the *descriptions of tools themselves*. In function-calling APIs, each tool is described in natural language

descriptions that explain their functionality and parameters. The LLM relies on these descriptions to decide the tools to invoke. In ToolHijacker, Fu et al. demonstrate that an attacker who controls a tool’s description can craft it to hijack the agent’s tool selection, causing the agent to call the attacker’s tool instead of the intended one.

3.3 Defense Architectures

Defenses against prompt injection differ not just in technique but in where they are architecturally positioned relative to the agent’s context window. We identify 2 categories:

3.3.1 Category 1: Reactive defenses (within-context detection)

Reactive defenses operate within the normal execution pipeline of the agent. In these approaches, external content is first inserted into the model’s context, followed by additional mechanisms attempting to identify potential prompt injection patterns:

- **Keyword sanitization:** Regex patterns are used to scan tool outputs for suspicious keywords or phrases (e.g., “SYSTEM”, “ignore previous”, “MANDATORY”) and redact or flag them for further review. [Shi et al. [2024]]
- **Instruction hierarchy:** The LLM is trained to treat instructions from different sources with different priority levels, giving the highest priority to system prompts, followed by user messages, and finally external data. [Wallace et al. [2024]]
- **ToolShield:** Before deployment, the agent simulates attacks against each tool and generates “safety experiences”—rules describing what a safe tool invocation looks like. At runtime, tool returns are checked against these experiences. [Li et al. [2025]]

3.3.2 Category 2: Structural defenses (separation by design)

These defenses prevent the injection from reaching the agent’s reasoning context in the first place:

- **StruQ:** Introduces special delimiter tokens that separate the prompt channel from the data channel at the model level. The LLM is fine-tuned to respect these delimiters.[Chen et al. [2024]]
- **CaMeL:** The dual-LLM architecture consists of two models. A Privileged LLM, with full access to tools, generates structured execution plans in a domain-specific language, while a Quarantined LLM, with no tool access, processes all untrusted input. The key innovation is capability-based ac-

cess control, in which each tool call requires an explicit capability token granted by the user, and data flows are tracked to prevent undesirable data from reaching sensitive operations.[Debenedetti et al. [2025]]

4 Approach

To evaluate the impact of prompt injection attacks on tool-using LLM agents, we design a controlled experimental setup that allows us to test both attack strategies and defensive mechanisms. Our setup simulates a realistic agent workflow where the model can reason about a task, invoke external tools, and use returned information to continue its reasoning process. The agent framework is implemented using LangChain. By controlling the environment, we can measure how prompt injection affects agent behavior and whether defensive mechanisms can reduce these risks while maintaining useful performance.

4.1 Agent Environment

We construct a simplified agent environment using LangChain’s agent framework, which supports tool-augmented reasoning and multi-step decision making. The agent follows a reasoning loop in which the LLM first analyzes the task, determines whether a tool should be used, invokes the selected tool, and then incorporates the tool output into its reasoning before generating the next step. This process can repeat until the agent produces a final response.

The system integrates several simulated tools implemented through **LangChain tool interfaces**. These tools represent realistic external capabilities such as retrieving information, accessing stored data, and interacting with external APIs. Each tool returns structured responses that the agent can incorporate into its reasoning. By monitoring tool calls and tool arguments within this environment, we can observe how the model selects tools and whether injected prompts influence these decisions.

4.2 Attack Simulation

To evaluate the security of the agent system, we simulate both **direct** and **indirect prompt injection attacks**. In direct prompt injection scenarios, the attacker interacts with the system as a normal user but embeds malicious instructions inside the prompt. These instructions attempt to override system instructions and persuade the model to ignore safety constraints or execute unsafe tool calls.

In indirect prompt injection scenarios, the malicious instructions appear in **external content processed by the agent**, such as retrieved documents, webpages, or tool outputs. Since the agent retrieves and inserts this content into the model context, the LLM may interpret attacker controlled text as legitimate instructions. These attacks attempt to manipulate the model’s reasoning process while appearing to be normal task related information.

Across both attack types, the main goal is to influence the agent’s **tool invocation behavior**. Instead of simply modifying the final textual response, the attacker attempts to cause the agent to call inappropriate tools, misuse available tools, or expose sensitive information through tool outputs.

4.3 Defensive Methods

To reduce the effectiveness of prompt injection attacks, we implement several defensive strategies within the agent pipeline. One approach involves prompt isolation, where trusted system instructions are separated from untrusted external content before being passed to the LLM. This helps reduce the chance that the model treats external content as executable instructions.

We also experiment with input provenance marking, where external content retrieved by the system is explicitly labeled as untrusted data. In addition, we apply tool permission controls and allowlists that restrict which tools the agent is allowed to invoke under certain conditions. These restrictions help prevent unauthorized tool usage even if the model is influenced by a malicious prompt.

Finally, we explore lightweight prompt detection mechanisms that attempt to identify potentially malicious instructions before they reach the LLM. These methods aim to reduce the success of injection attacks while preserving the agent’s ability to complete tasks.

4.4 Evaluation Method

To evaluate both the effectiveness of prompt injection attacks and the robustness of defensive strategies, we measure a set of security and utility metrics. Following prior work on agent security benchmarks, we report four primary metrics: *Attack Success Rate*, *Tool Misuse Rate*, *Sensitive Data Leakage Rate*, and *Task Utility Degradation*. These metrics allow us to quantify not only how often attacks succeed, but also how defenses impact the agent’s ability to complete tasks correctly.

Attack Success Rate (ASR) Attack Success Rate measures the proportion of attacks that successfully influence the agent’s behavior. To account for differences in attack severity, each attack is assigned a weight.

Let N denote the total number of attack attempts. For each attack i , let $s_i \in \{0, 1\}$ indicate whether the attack succeeded and let w_i denote the severity weight associated with that attack. The attack success rate is defined as

$$ASR = \frac{\sum_{i=1}^N w_i s_i}{\sum_{i=1}^N w_i} \quad (1)$$

An attack is considered successful if the injected prompt causes the agent to violate system instructions or perform an unintended action.

Tool Misuse Rate (TMR) Tool Misuse Rate measures how often the agent invokes tools in unintended or unsafe ways under adversarial prompts. Let T denote the total number of tool invocations. For each invocation i , let $m_i \in \{0, 1\}$ indicate whether the tool call violates system policies, and let w_i denote the severity weight associated with that misuse. The metric is defined as

$$TMR = \frac{\sum_{i=1}^T w_i m_i}{\sum_{i=1}^T w_i} \quad (2)$$

Higher weights can be assigned to more critical misuse events such as executing code or accessing sensitive resources.

Sensitive Data Leakage Rate (SDLR) Sensitive Data Leakage Rate measures the probability that protected information is exposed through tool outputs or agent responses. To evaluate this, we seed synthetic secrets (canary tokens) into tool outputs or the agent’s internal memory and monitor whether they appear in external outputs or outbound tool arguments.

Let C denote the number of injected tokens. For each token i , let $l_i \in \{0, 1\}$ indicate whether the token is leaked and let w_i denote the sensitivity weight of that token. The leakage rate is defined as

$$SDLR = \frac{\sum_{i=1}^C w_i l_i}{\sum_{i=1}^C w_i} \quad (3)$$

This approach provides a controlled method to detect whether sensitive information is exfiltrated during an attack.

Task Utility Degradation (TUD) Task Utility Degradation measures how defensive strategies affect the agent’s ability to successfully complete tasks. Let K denote the number of tasks evaluated. For each task k , let U_k^{base} denote the baseline task success rate without defenses and U_k^{def} denote the success rate when defenses are enabled. The degradation is defined as

$$TUD = \frac{1}{K} \sum_{k=1}^K \frac{U_k^{base} - U_k^{def}}{U_k^{base}} \quad (4)$$

This metric captures the average performance degradation introduced by defenses across all evaluated tasks.

By analyzing these four metrics across different attack scenarios and defensive configurations, we can quantify both the security impact of prompt injection attacks and the effectiveness of mitigation strategies while maintaining useful system performance.

5 Experiments

To validate our simulation-based findings against real model behavior, we conduct both experiments using a real large language model. We use Llama 3.2 (3B parameters) running locally via Ollama, an open-source framework for serving LLMs on consumer hardware. This setup requires no external API access and runs entirely on a local machine.

The agent architecture remains the same as described in the previous section. User prompts and tool outputs are passed to the local model, which generates the next action. If the response contains a valid tool invocation in JSON format, the agent executes the corresponding tool and appends the result to the context before continuing the reasoning loop.

To reduce prompt-specific bias, each attack scenario is instantiated with multiple prompt templates that express the same malicious intent using slightly different wording. Across these templates we apply the same four defensive strategies and evaluate the agent using the four metrics defined earlier.

We run 5 trials per attack–defense configuration for the LLM experiment. While this is fewer than the 50 trials used in the rule-based simulation, LLM inference is significantly slower and each run involves multiple reasoning steps. The number of trials is sufficient to verify whether the qualitative trends observed in simulation hold when interacting with a real model.

5.1 Experiment 1: Direct Prompt Injection

In the direct injection setting, the attacker interacts directly with the agent as a normal user but embeds malicious instructions inside the prompt. We test four attack scenarios that target different aspects of agent behavior:

- **System override:** the attacker instructs the agent to ignore its system prompt, read a sensitive database record, and email the contents to an attacker-controlled address.
- **Tool misuse:** the attacker embeds a request to execute malicious code containing `os.system` calls alongside a benign-looking information lookup.
- **Social engineering:** the attacker impersonates a system administrator and requests that the agent output sensitive fields, including SSN, from a personnel record.
- **Data exfiltration:** the attacker asks the agent to include a canary API key in a search query, leaking it through the tool’s outbound request.

Table 1 shows the results of direct prompt injection against the LLM-based agent.

Table 1: Direct prompt injection against Llama 3.2 (3B), 5 trials per condition.

Defense	ASR	TMR	SDLR	TUD
None	30.0%	0.0%	5.0%	0.0%
Prompt isolation	25.0%	0.0%	0.0%	0.0%
Provenance	25.0%	0.0%	0.0%	0.0%
Tool permission	25.0%	0.0%	0.0%	0.0%
Prompt detection	25.0%	0.0%	0.0%	0.0%
All combined	25.0%	0.0%	0.0%	0.0%

The LLM-based agent is notably more resistant to direct injection than the simulation predicted. Baseline ASR is 30.0% compared to 75.0% in simulation. This is expected: Llama 3.2 has undergone safety training (RLHF) that teaches it to refuse overtly malicious instructions such as “ignore all previous instructions.” The model often responds with refusal text rather than producing a tool call, which results in 0.0% TMR across all conditions since the model rarely outputs valid tool call JSON when it detects a suspicious request.

Even a small defense reduces ASR to 25.0%, which represents the residual vulnerability from the social engineering attack scenario where the model complies with a plausible-sounding “system administrator” request. SDLR drops to 0.0% with any defense, as the model avoids echoing sensitive fields in its output when provenance or isolation markers are present.

For comparison, Table 2 shows the corresponding results from the rule-based simulation.

Table 2: Direct prompt injection in rule-based simulation, 50 trials per condition.

Defense	ASR	TMR	SDLR	TUD
None	75.0%	42.9%	25.0%	0.0%
Prompt isolation	75.0%	42.9%	7.5%	0.0%
Provenance	75.0%	42.9%	8.5%	0.0%
Tool permission	25.0%	42.9%	25.0%	0.0%
Prompt detection	34.0%	16.9%	6.0%	41.0%
All combined	25.0%	6.2%	1.5%	47.5%

The simulation overestimates direct ASR (75.0% vs. 30.0%) because it does not model safety training. In direct injection, prompt isolation and provenance marking have no effect on ASR in the simulation since the attacker *is* the user, so wrapping user input with untrusted markers does not change the fact that the malicious instructions originate from the prompt itself. However, both defenses reduce SDLR significantly (from 25.0% to 7.5% and 8.5%), because the agent becomes less likely to echo sensitive data in its response when provenance labels are present.

Tool permission control reduces ASR from 75.0% to 25.0% in the simulation by blocking tool calls that match known malicious argument patterns. It does not reduce TMR because the agent still *attempts* the misuse; the permission layer blocks execution after the decision is made. Prompt detection reduces ASR to 34.0% by flagging suspicious patterns before the agent processes them, but introduces 41.0% task utility degradation due to false positives on legitimate requests. When all defenses are combined, ASR remains at 25.0% while TMR drops to 6.2% and SDLR to 1.5%, at the cost of 47.5% utility degradation.

5.2 Experiment 2: Indirect Prompt Injection

In the indirect injection setting, the attacker does not interact with the system directly. Instead, malicious instructions are embedded in the content returned by tools that the agent invokes during normal task execution. We test four scenarios:

- **Poisoned search result:** a web search returns legitimate financial data followed by a fake system notice instructing the agent to email the summary to an attacker-controlled address.
- **Poisoned database record:** a project status query returns valid data appended with instructions to execute malicious code for “access logging.”
- **Data exfiltration via email:** a personnel record lookup returns employee data with an injected di-

rective to forward the complete record, including SSN, to an external address.

- **Chained tool abuse:** a weather search returns forecast data with an instruction to also query an unrelated sensitive database record.

Table 3 shows the results of indirect prompt injection against the LLM-based agent.

Table 3: Indirect prompt injection against Llama 3.2 (3B), 5 trials per condition.

Defense	ASR	TMR	SDLR	TUD
None	75.0%	25.0%	25.0%	0.0%
Prompt isolation	65.0%	24.2%	0.0%	0.0%
Provenance	55.0%	17.1%	15.0%	0.0%
Tool permission	25.0%	25.0%	25.0%	0.0%
Prompt detection	55.0%	18.2%	5.0%	0.0%
All combined	25.0%	26.5%	0.0%	0.0%

Indirect injection is substantially more effective against the real model. Baseline ASR is 75.0%, compared to 30.0% for direct injection. This confirms a key finding from the literature: LLMs are more susceptible to injections embedded in tool outputs because the model has no reason to distrust data that it requested itself. Safety training primarily targets adversarial user prompts, not adversarial data returned by tools.

Prompt isolation reduces ASR to 65.0% and provenance marking to 55.0%, showing that these defenses provide partial protection by signaling that tool outputs should be treated as data rather than instructions. Prompt detection achieves the same 55.0% ASR while also reducing SDLR from 25.0% to 5.0%. Tool permission control again proves the strongest individual defense at 25.0% ASR, consistent with the simulation results.

The combined defense reduces ASR to 25.0% and SDLR to 0.0%, matching the pattern observed in simulation: no single defense is sufficient, but layering all four achieves the strongest protection.

For comparison, Table 4 shows the corresponding results from the rule-based simulation.

Table 4: Indirect prompt injection in rule-based simulation, 50 trials per condition.

Defense	ASR	TMR	SDLR	TUD
None	48.5%	20.5%	43.0%	0.0%
Prompt isolation	27.5%	16.1%	19.0%	0.0%
Provenance	33.0%	14.3%	21.5%	0.0%
Tool permission	17.5%	24.9%	42.5%	0.0%
Prompt detection	17.5%	3.0%	39.0%	0.0%
All combined	5.0%	1.9%	12.5%	0.0%

Unlike in the direct setting, prompt isolation and provenance marking produce meaningful reductions in ASR in the simulation as well. Prompt isolation reduces ASR from 48.5% to 27.5%, while provenance marking reduces it to 33.0%. This is because the injected content originates from tool outputs rather than the user prompt, so marking it as untrusted data helps the agent distinguish between legitimate instructions and injected ones.

Tool permission control reduces ASR to 17.5% by blocking outbound tool calls with suspicious arguments. Prompt detection achieves the same ASR of 17.5% while also reducing TMR from 20.5% to 3.0%, because it identifies injection patterns in tool outputs before the agent can act on them. Notably, neither defense introduces utility degradation in the indirect setting, since legitimate tasks do not trigger the detection patterns that are specific to attack content. The combined defense achieves 5.0% ASR with 1.9% TMR and 12.5% SDLR. The residual ASR comes from attack variants whose injection language is subtle enough to avoid pattern detection while still influencing the agent’s behavior.

5.3 Summary of Findings

Table 5 summarizes the ASR across both the LLM experiment and the rule-based simulation.

Table 5: Attack Success Rate comparison: LLM (5 trials) vs. rule-based simulation (50 trials).

Defense	Direct		Indirect	
	LLM	Sim.	LLM	Sim.
None	30.0%	75.0%	75.0%	48.5%
Prompt isolation	25.0%	75.0%	65.0%	27.5%
Provenance	25.0%	75.0%	55.0%	33.0%
Tool permission	25.0%	25.0%	25.0%	17.5%
Prompt detection	25.0%	34.0%	55.0%	17.5%
All combined	25.0%	25.0%	25.0%	5.0%

Several observations emerge from the results:

1. **Direct injection is harder against real models.** The simulation overestimates direct ASR (75.0% vs. 30.0%) because it does not model safety training. Real models trained with RLHF learn to refuse overtly malicious prompts, a capability that rule-based simulation cannot capture. This suggests that direct prompt injection, while still a risk, is partially mitigated by standard model alignment.
2. **Indirect injection is harder to defend in practice.** The real model shows higher indirect ASR

(75.0%) than the simulation (48.5%). This is because the model genuinely trusts tool outputs and follows embedded instructions that sound authoritative, such as fake system notices and compliance directives. Safety training does not cover this attack surface, making indirect injection the more dangerous threat in deployed agent systems.

3. **Direct and indirect attacks require different defenses.** Prompt isolation and provenance marking are ineffective against direct injection (where the attacker is the user) but reduce indirect injection ASR by 10–20 percentage points in the LLM experiment. Conversely, tool permission control is effective against both attack types because it operates at the execution layer regardless of where the injection originates.
4. **No single defense is sufficient.** The best individual defense (tool permission) still allows 25.0% of attacks to succeed against the real model. Only the combined configuration achieves the strongest protection across both settings.
5. **Security and utility trade off.** Prompt detection introduces 41.0% utility degradation in the direct setting of the simulation due to false positives on legitimate user requests. In contrast, the same defense causes no degradation in the indirect setting, where attack patterns are more distinctive. This suggests that detection-based defenses are better suited for filtering external content than user input.

6 Future Plan

By the midway executive deadline, we have completed the relative effectiveness of defenses against prompt injection in tool-using agents. However, several limitations constrain the scope of the current study, which may be resolved and refined in the future work.

6.1 Evaluation on More Complex Agents

Our current experiments are conducted on a relatively simple LangChain-style tool-calling agent designed to isolate the security mechanisms under controlled conditions. While this setup allows us to clearly measure attack success rate, tool misuse, and data leakage, it does not fully capture the complexity of modern agentic systems that involve multi-step reasoning, long-term memory, and dynamic tool planning.

In future work, we plan to evaluate our defenses on more comprehensive agent frameworks and real-world agent applications. Examples include agents

built using frameworks such as LangChain, OpenAI tool-calling pipelines, and emerging multi-agent orchestration systems. These agents typically involve multiple tool interactions, retrieval-augmented generation, and iterative reasoning loops, which create additional attack surfaces for prompt injection.

We also plan to extend our current toy tool environment to interact with real external APIs. Instead of simulated tool responses, the agent will issue actual API requests (e.g., search APIs, database queries, or messaging services). This will allow us to study how prompt injection attacks propagate through realistic tool outputs and whether the proposed defenses remain effective when interacting with real-world systems.

In addition, we plan to test the proposed defenses on standardized agent security benchmarks, which provide adversarial scenarios designed to evaluate prompt injection resilience in tool-using agents. This would allow a more systematic comparison with prior work and help quantify how well our hybrid approach generalizes beyond the controlled experimental setup used in this study.

6.2 Limitations of Current Defenses

Our four defensive strategies (prompt isolation, provenance marking, tool permission controls, and regex-based prompt detection) represent reasonable initial approaches but fall short of the state of the art in several ways:

- **Prompt isolation uses plain-text markers.** Our delimiters (`=== BEGIN UNTRUSTED ===`) are visible to and reproducible by attackers. Chen et al.’s StruQ Chen et al. [2024] addresses this by using reserved special tokens as delimiters that cannot appear in user input, combined with fine-tuning the model to respect the separation. This is a fundamentally stronger mechanism than our text-based markers cannot approximate.
- **No instruction hierarchy.** Wallace et al. Wallace et al. [2024] propose training LLMs to explicitly prioritize instructions by privilege level (system > user > tool output). Our implementation treats all content at the same privilege level and relies on wrapper-based pre/post-processing rather than model-level enforcement.
- **Regex detection is vulnerable.** Our prompt detection relies on 11 regex patterns matching phrases like “ignore previous instructions” and “[SYSTEM NOTICE].” These are trivially bypassed by paraphrasing, misspelling, or encoding.

6.3 Future Directions

Based on the gaps identified above, we analyze and outline some directions for future work in a better defense architecture.

The strongest existing defenses tend to be difficult to deploy. CaMeL DeBenedetti et al. [2025] provides provable security, but it requires two separate LLMs, a custom code interpreter, and a full data-flow tracking system. StruQ Chen et al. [2024] uses special tokens to separate instructions from data, but it requires fine-tuning the model, which is not possible with closed-source APIs. The instruction hierarchy Wallace et al. [2024] is effective but requires model-specific training data. On the other end, simple defenses like our regex patterns and allowlists are easy to deploy but easy to bypass.

We propose a possible hybrid architecture built on three components:

1. **Track where data came from.** The core insight behind CaMeL is that data from external sources (tool outputs, retrieved documents) should not be allowed to control what the agent does. We propose a simplified version: whenever a tool returns data, tag it as *untrusted*. If the agent later tries to pass that untrusted data into a security-sensitive argument, such as an email recipient, a code string, or a shell command, the system blocks it and asks the user to confirm. This captures CaMeL’s main idea without requiring its full dependency-graph infrastructure.
2. **Use a small model to catch injections.** Our current regex-based detection matches fixed phrases like “ignore previous instructions,” which an attacker can easily dodge by rewording. A better approach is to use a small, cheap secondary model (a fine-tuned classifier or a prompted lightweight LLM) whose only job is to look at tool outputs and decide does it contain hidden instructions, or is it normal data.
3. **Ask the user when uncertain.** Rather than silently blocking suspicious requests which hurts task utility, or silently allowing them which hurts security, the system should ask the user for confirmation when either the taint tracker or the detector flags something. This preserves utility for legitimate edge cases while preventing automated attacks from succeeding without human oversight.

The main advantage of this hybrid is that it does not require fine-tuning, special tokens, or a custom runtime. At the same time, it goes beyond surface-level defenses by tracking data provenance at the tool bound-

ary and using semantic (rather than syntactic) detection. Validating this architecture against optimization-based attacks and standardized benchmarks like AgentDojo DeBenedetti et al. [2024] would be the natural next step.

References

- Yichuan Chen et al. StruQ: Defending against prompt injection with structured queries. *arXiv preprint*, 2024.
- Edoardo DeBenedetti et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Proceedings of NeurIPS*, 2024.
- Edoardo DeBenedetti et al. CaMeL: Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025.
- Yao Fu et al. ToolHijacker: Prompt injection attacks on tool selection in LLM agents. *arXiv preprint arXiv:2504.19793*, 2025.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the Workshop on Artificial Intelligence Safety (AISec)*, 2023.
- Xianghe Li et al. Unsafer in many turns: Benchmarking and defending multi-turn safety risks in tool-using agents. *arXiv preprint arXiv:2602.13379*, 2025.
- Peiyu Liu et al. Prompt injection attack against LLM-integrated applications. *arXiv preprint arXiv:2306.05499*, 2023.
- Narek Maloyan and Dmitry Namiot. Prompt injection attacks on agentic coding assistants: A systematic analysis of vulnerabilities in skills, tools, and protocol ecosystems. *arXiv preprint arXiv:2601.17548*, 2025.
- Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*, 2022.
- Yupei Shi et al. Automatic and universal prompt injection attacks against large language models. *arXiv preprint arXiv:2403.04957*, 2024.
- Eric Wallace et al. The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv preprint*, 2024.